

FIT3155: Week 8 Lab Sheet

(Questions are based on Week 7 topic dealing with Fibonacci heaps.)

Objectives: Develop a working understanding of the structure and operations of Fibonacci heaps, leading to how they support efficient amortized time for priority queue operations.

Conceptual exercises

1. You studied Dijkstra's algorithm in FIT2004. Suppose you are given a general heap data structure that supports the operations **extract-min** and **decrease-key**. Express the overall time complexity of Dijkstra's algorithm on a graph $G(V, E, W)$ in terms of:

Ans: 1

Each iteration of Dijkstra's algorithm maintains a partition of the vertices into two sets: C and $V \setminus C$. The set C contains vertices whose shortest distances from the source have been finalized, whereas $V \setminus C$ contains vertices whose distances are still unfinalized 'best-estimates' thus far, and are maintained in a priority queue using a heap data structure.

In each iteration, the algorithm performs an **extract-min** operation on that heap to select a vertex u with the minimum distance estimate, and then adds u to C . For each edge $u \rightarrow v$, a relaxation step is performed. If a shorter path to v is found via u , and $v \in V \setminus C$, the distance estimate of v is revised using a **decrease-key** operation.

Initially, the distance estimates for all vertices are set to ∞ , except for the source vertex s , which is set to 0. The algorithm continues this process until all vertices have been moved into C .

Throughout the algorithm, **extract-min** is performed once per vertex, for a total of $|V|$ times. Each edge relaxation can lead to at most one **decrease-key** operation, and only when a shorter path is found, so there are $O(|E|)$ such operations overall. Therefore, the time complexity of Dijkstra's algorithm can be expressed as $O(|V| \cdot T_{\text{extract-min}} + |E| \cdot T_{\text{decrease-key}})$.

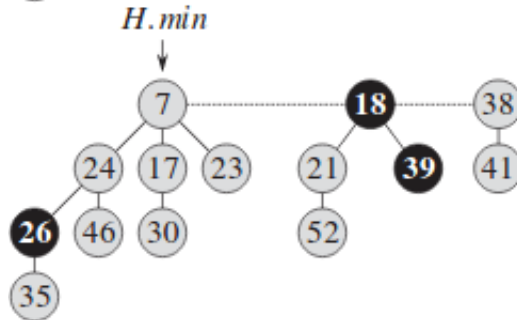
Additional comments: If a binary heap is used (as you may recall from FIT2004), $T_{\text{extract-min}}$ and $T_{\text{decrease-key}}$ are both $O(\log |V|)$, giving a total time complexity of $O((|V| + |E|) \log |V|)$. If Fibonacci heaps are used instead, the amortized complexity of **extract-min** is $O(\log |V|)$ and of **decrease-key** is $O(1)$, giving a total (amortized) time complexity of $O(|V| \log |V| + |E|) = O(|E| + |V| \log |V|)$, which is asymptotically more efficient than the binary heap implementation.

2. Insert the following elements into a Fibonacci heap:

3, 8, 10, 11, 1, 12, 2, 17

and then perform **extract-min** on the resulting heap.

3. Perform **extract-min** on the following Fibonacci heap:



Ans: 2-3

The hand-worked solutions* for these questions are attached.

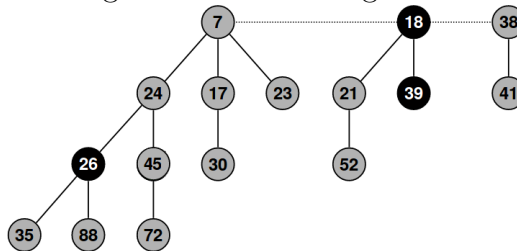
4. What are the worst-case time complexities of **insert** and **extract-min** operations of a Fibonacci heap?

Ans: 4

The **insert** has worst-case time complexity $O(1)$. This is because any element is directly inserted to the left of *H.min* into the root list that is maintained as a circular doubly linked list.

The **extract-min** operation has worst-case time complexity $O(N)$, where N is the current size of the heap. This is because, in the worst case, all N elements may be in the root list and must be traversed during the consolidation step to ensure that there is at most one tree of any degree.

5. Starting from the following state of a Fibonacci heap:



Run the following sequence of operations, and after each step, draw the resultant heap:

- decrease-key of 45 to 40.
- decrease-key of 40 to 12.
- decrease-key of 35 to 1.
- extract-min**.

Ans: 5

The hand worked solution* for this question is attached.

*Credit: Michael Gill

6. Let F_n denote the n -th Fibonacci number, where $F_n = F_{n-1} + F_{n-2}$. Prove using induction that $\sum_{i=0}^n F_i = F_{n+2} - 1$.

Ans: 6

For $n = 0$, L.H.S. $\sum_{i=0}^0 F_i = F_0 = 0$. R.H.S. $F_{0+2} - 1 = 1 - 1 = 0$, so the statement holds.

Assume that the statement holds for all integers $\leq k$.

Then, for $k + 1$, we have

$$\begin{aligned} \sum_{i=0}^{k+1} F_i &= \sum_{i=0}^k F_i + F_{k+1} \\ &= (F_{k+2} - 1) + F_{k+1} \quad (\text{by inductive assumption}) \\ &= \underbrace{F_{k+2} + F_{k+1}}_{=F_{k+3}} - 1 \\ &= F_{k+3} - 1. \end{aligned}$$

Therefore, by induction, the statement holds for all $n \geq 0$.

7. Using induction, show that for all integers $n \geq 0$, the $(n + 2)$ -th Fibonacci number satisfies $F_{n+2} \geq \phi^n$, where $\phi = \frac{1+\sqrt{5}}{2}$ (aka, the Golden Ratio).

Ans: 7

For $n = 0$, L.H.S. $F_2 = 1$ and R.H.S. $\phi^0 = 1$, so the statement holds.

For $n = 1$, L.H.S. $F_3 = 2$, which is greater than R.H.S. $\phi^1 \approx 1.618$. Therefore the statement again holds.

Assume that $F_{i+2} \geq \phi^i$ holds for all integers $\leq k$.

Then for $k + 1$, we have on the L.H.S. $F_{k+1} + 2 = F_{k+3}$. Expanding

$$\begin{aligned} F_{k+3} &= F_{k+2} + F_{k+1} \\ &\geq \phi^k + \phi^{k-1} \quad (\text{by inductive assumption}) \\ &\geq \phi^{k-1} \underbrace{(\phi + 1)}_{=\phi^2} \quad (\text{see note below}) \\ &\geq \phi^{k-1} \phi^2 \\ &\geq \phi^{k+1} \end{aligned}$$

(Note: Above we used the identity $\phi^2 = \phi + 1$, which comes from the fact that ϕ is the +ve root of the quadratic equation $x^2 - x - 1 = 0$; refer non-examinable proof uploaded in the Week 7 section on moodle.)

Thus, by induction, $F_{n+2} \geq \phi^n$ for all $n \geq 0$.

8. Prove the maximum degree bound of $O(\log N)$ on any node in a Fibonacci Heap containing N elements.

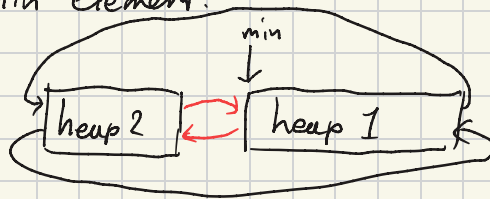
Ans: 7

See proof uploaded in the Week 7 section on moodle.

Problem 2: Insert the following list into a fibonacci heap.

3, 8, 10, 11, 1, 12, 2, 17.

Given two fibonacci heaps we are able to merge them by inserting heap 2 to the left of the heap 1 min element.



Taking each element as its own fib heap gives:

8 3

*links omitted for clarity.

8 10 3

8 10 11 3

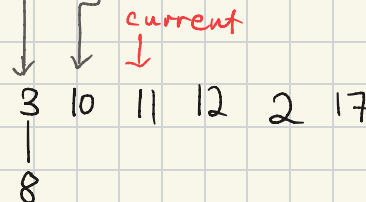
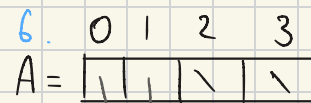
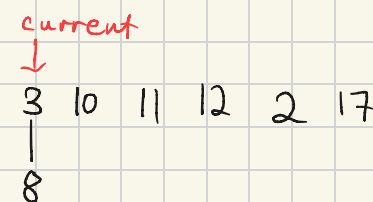
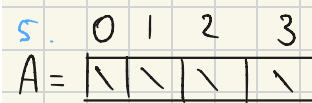
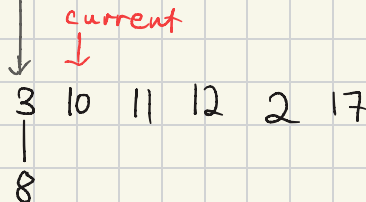
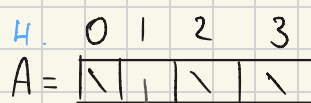
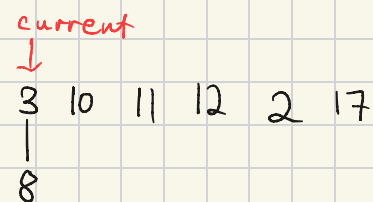
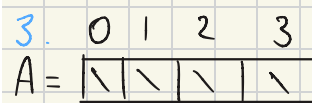
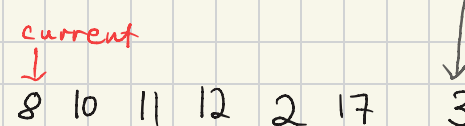
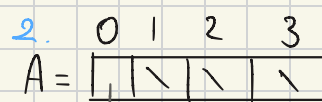
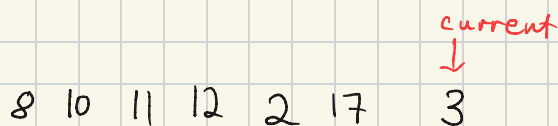
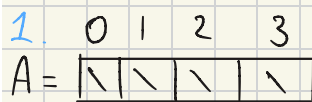
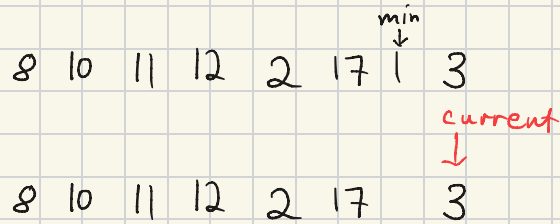
8 10 11 1 3

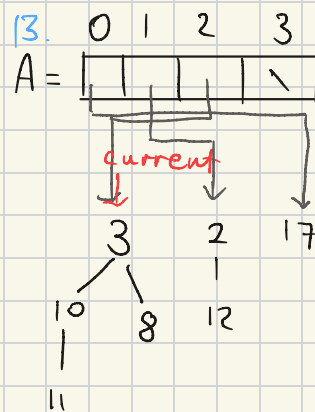
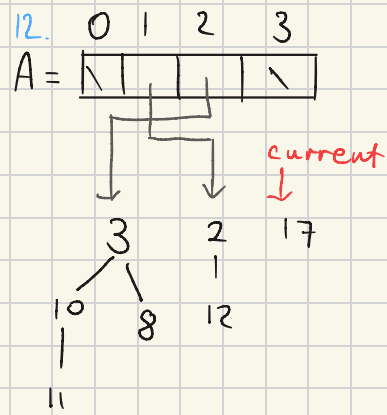
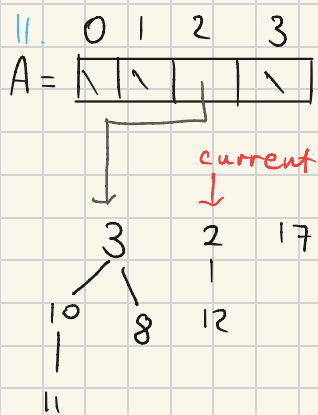
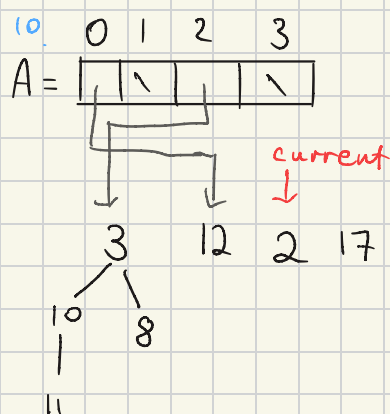
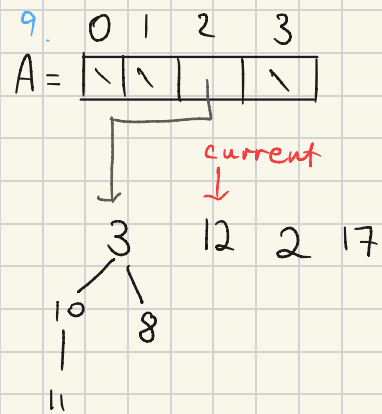
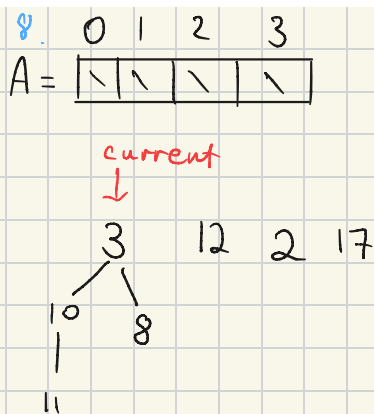
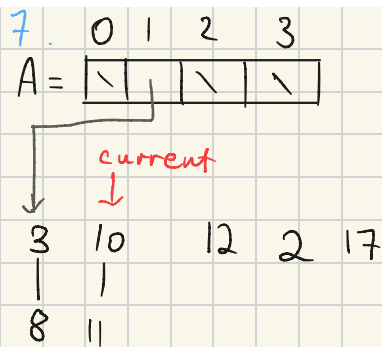
8 10 11 12 1 3

8 10 11 12 2 1 3

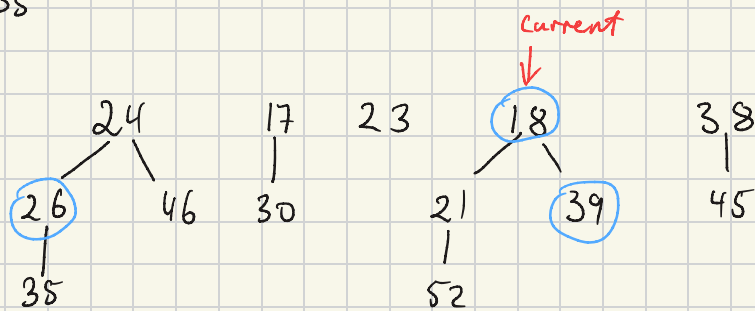
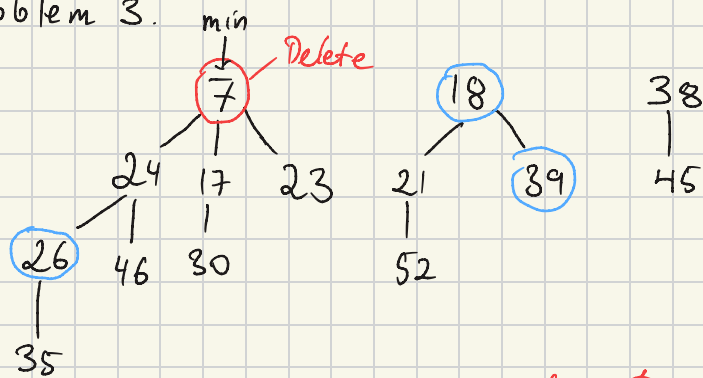
8 10 11 12 2 17 1 3

To perform extract min, we first remove the min element and merge the children of the min with the heap before consolidating.

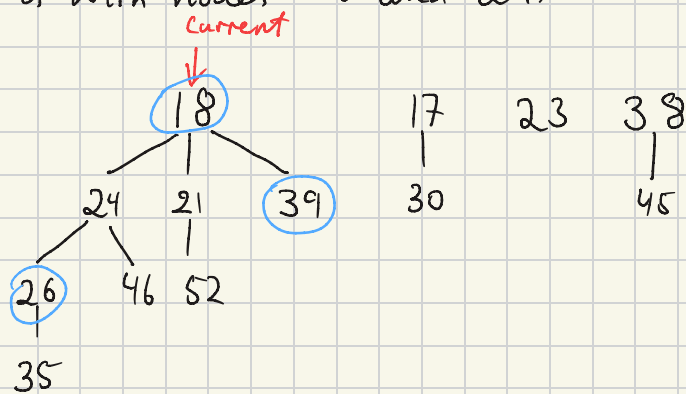




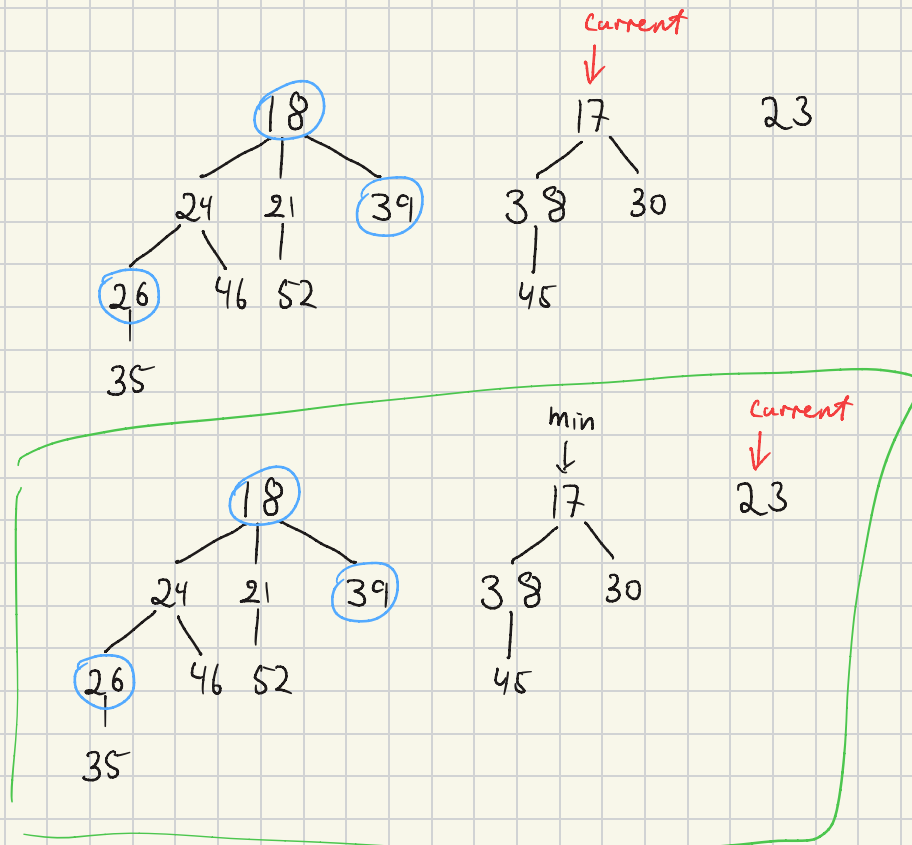
Problem 3.



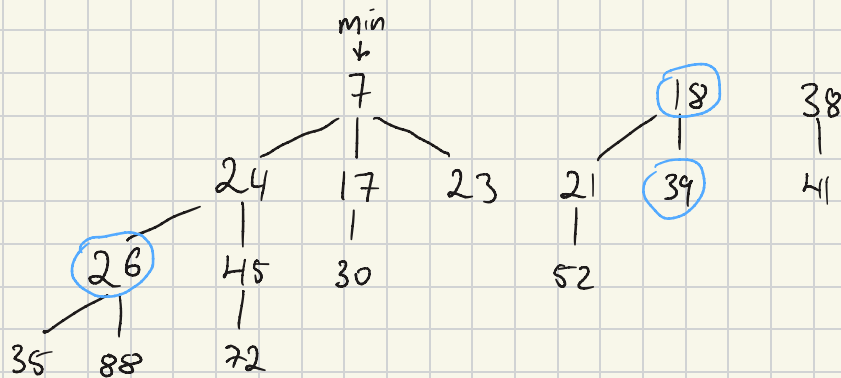
Following the process from P1, the first consolidation is with nodes 18 and 24.



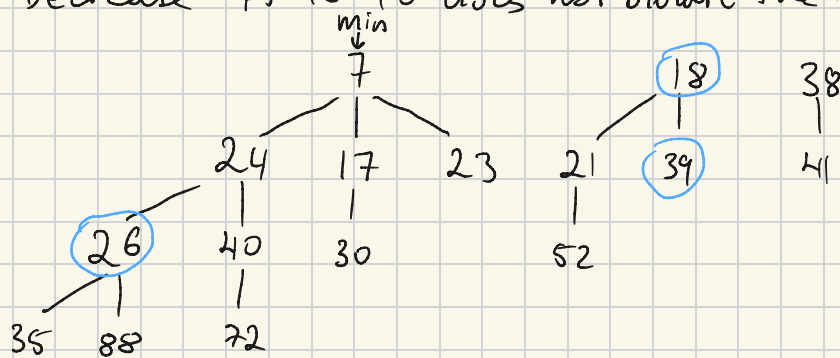
The second consolidation happens with nodes 38 and 17.



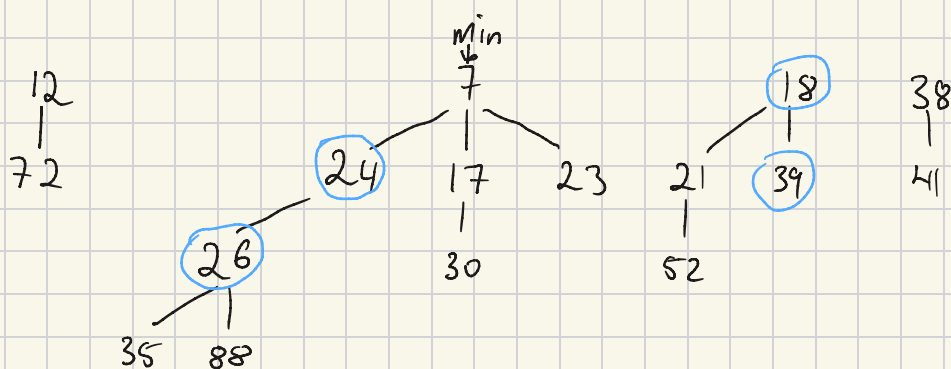
Problem 4.



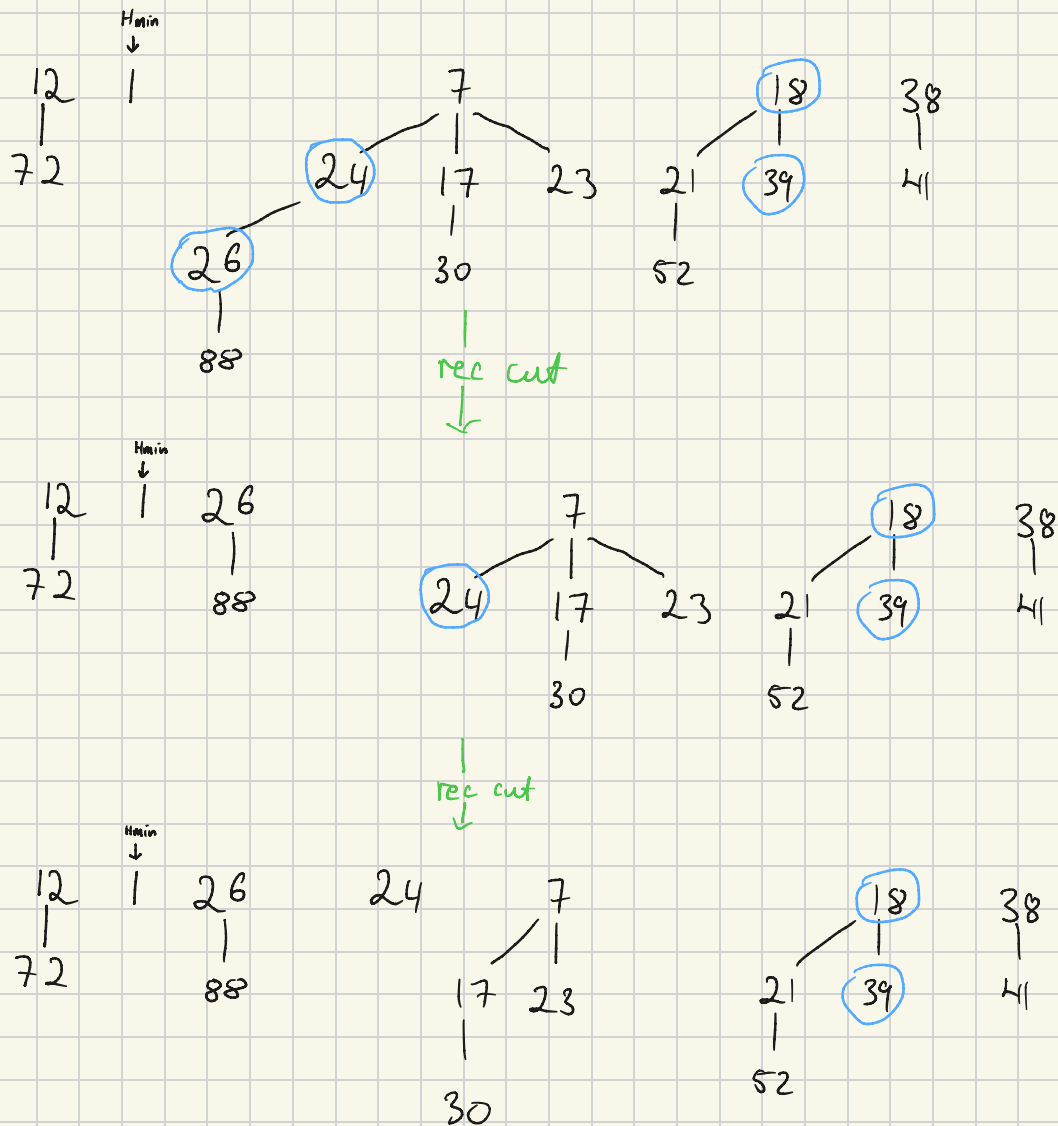
Decrease 45 to 40 does not violate the heap property

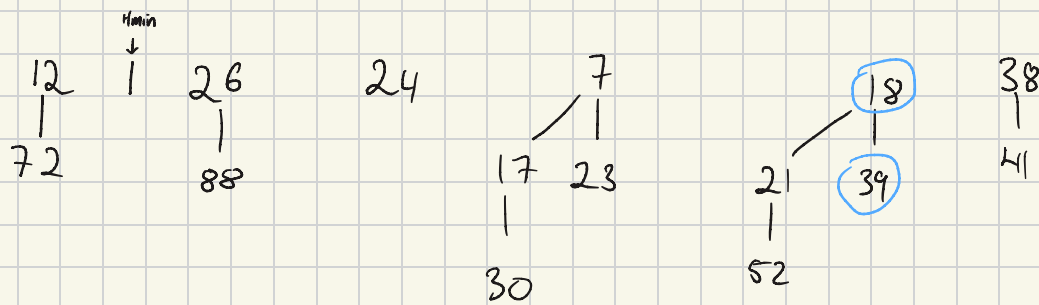


Decrease 40 to 12 violates the heap property, however 24 is unmarked so we only need to cut 12 and mark 24.

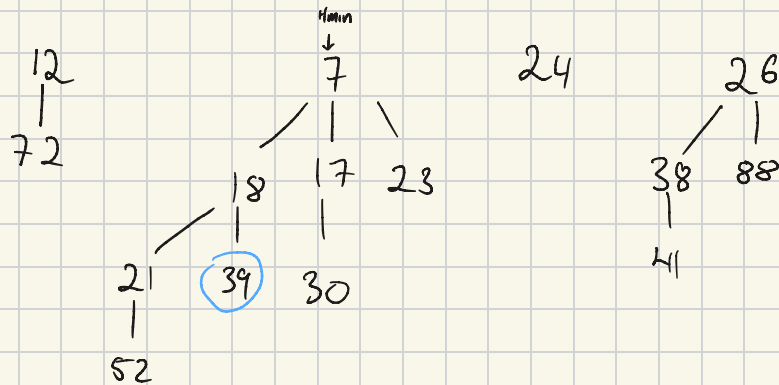


Decreasing 35 to 1 also violates the heap property. This time the parent is marked, so we must recursively cut. If a marked node is cut then it must be unmarked.





We now perform extract min. Following the steps in Problem 2 we get:



* Note that 18 is unmarked as it is made a child